```
in data cleaning, the paper promotes tidy data as a
crucial step towards more efficient and accessible
statistical analysis.\n`
}
*/
```

In this listing, the `loadSummarizationChain` function takes the model as the first argument and an options object as the second argument, where the `type` is set to `"map_reduce"` and `verbose` is set to `true` to enable verbose logging.

The returned object then is used to invoke the summarization chain by calling the `invoke` method, passing in an object with the `input_documents` property, which contains the documents to be summarized.

Once the process is complete (which, depending on the length of the document and the network connection, could take a few seconds), the final result is printed to the console.

> **NOTE** I've added another example using the refine methodology in the file located at summarize-refine.js inside the ch07/summarization-examples folder.

> **WARNING** Summarizing large documents will cost you credits. We provide an example folder with some PDF documents that are fairly small, but if you want to summarize larger documents, you should be aware of the cost implications. Using the GPT-3.5 Turbo model is generally cheaper than using GPT-4 or later models. Additionally, I found that the Google AI provider is often more reliable and generous in terms of summarizing text compared to the OpenAI provider. Be sure to keep track of your credit usage when summarizing longer documents using either the MapReduce or the refine methods. You can typically monitor your usage and spending directly through the respective AI provider's online dashboard (e.g., OpenAI's Usage Dashboard, Google Cloud's billing section for Gemini API).

Before we showcase the application, we spend some time explaining the main challenges and limitations when it comes to document summarization using LLMs.

7.1.4 Caveats and limitations of document summarization

While our current examples of summarizing documents are effective for relatively small texts, there are several nuances to consider when addressing edge cases.

Parsing and cleaning up documents

The initial step of parsing documents can be entangled with difficulties, especially when dealing with complex formats like PDFs or DOCX files. Text may be scattered across different sections, embedded in images, or poorly formatted, leading to incomplete or inaccurate data extraction. For instance, in PDFs, text can be encoded in various ways, making it challenging to extract the content accurately. Elements like tables, footnotes, and annotations can complicate the parsing process further.